

全连接网络实现鸢尾花数据分类 实验指导

实验概览

实验目的

- 理解全连接神经网络（FCNN）的基本原理和结构。
- 掌握使用全连接神经网络对鸢尾花数据集进行分类的方法。
- 学习如何评估全连接神经网络的分类性能，并调整网络参数以优化性能。。
- 理解验证集在模型调参中的作用，掌握通过交叉验证选择最优超参数的方法。

实验要求

- 熟悉 Python 及常用深度学习库。
- 了解神经网络的基本概念、训练过程和评估方法。
- 具备基础的机器学习知识，特别是分类任务的相关理论。

提交内容

实验报告（包含实验目的、步骤、结果分析等），采用 PDF 格式。报告应结构清晰、内容详实、语言流畅。

全连接神经网络

全连接神经网络（Fully Connected Neural Network，**FCNN**）是一种基本的神经网络结构，其中每一层的**每个**神经元都与下一层的**所有**神经元相连。这种结构使得网络能够学习输入数据中的复杂模式，并进行**分类或回归**等任务。

网络结构

全连接神经网络通常由**输入层**、**多个隐藏层**和**输出层**组成。输入层接收原始数据，隐藏层对数据进行特征提取和转换，输出层给出最终的分类结果。

激活函数

激活函数是神经网络中的重要组成部分，它决定了神经元是否应该被激活。常用的激活函数包括 **ReLU**、**sigmoid** 和 **tanh** 等。

损失函数与优化器

损失函数用于衡量网络预测结果与真实标签之间的差异，优化器则用于根据损失函数的梯度来更新网络的权重。

常用的损失函数包括交叉熵损失，常用的优化器包括**随机梯度下降（SGD）**和 **Adam** 等。

全连接神经网络的应用

全连接神经网络广泛应用于图像识别、语音识别、自然语言处理等领域，是深度学习中的基础模型之一。

全连接网络实现鸢尾花数据分类

1.数据准备

(1) 加载数据集

加载鸢尾花数据集，该数据集包含 150 个样本，每个样本有 4 个特征（花萼长度、花萼宽度、花瓣长度、花瓣宽度），分为 3 个类别。

(2) 数据预处理

对数据进行预处理，如归一化或标准化，以提高网络的训练效果。

(3) 划分数据集

将数据集划分为训练集（80%）和测试集（20%）。

2.网络构建

(1) 定义网络结构

构建全连接神经网络，设置合适的网络结构，包括输入层、隐藏层和输出层。（网络层的神经元数量可自行调整）

```
def __init__(self, input_size=4, hidden_size1=64, hidden_size2=32, output_size=3):
    # input_size: 输入特征维度，鸢尾花数据集为4个特征
    # hidden_size1: 第一个隐藏层的神经元数量
    # hidden_size2: 第二个隐藏层的神经元数量
    # output_size: 输出层维度，鸢尾花有3个类别
    super(IrisNet, self).__init__()
    self.fc1 = nn.Linear(input_size, hidden_size1)    # 第一层全连接
    self.fc2 = nn.Linear(hidden_size1, hidden_size2)  # 第二层全连接
    self.fc3 = nn.Linear(hidden_size2, output_size)   # 第三层全连接
    self.relu = nn.ReLU()                             # 激活函数
    self.dropout = nn.Dropout(0.2)                   # Dropout层，丢弃概率为20%，防止过拟合
```

(2) 选择激活函数

隐藏层一般采用 ReLU 作为激活函数，输出层一般采用 softmax 作为激活函数。

(3) 定义损失函数和优化器

定义交叉熵损失函数和 Adam 优化器，设置学习率。

```
criterion = nn.CrossEntropyLoss()                # 交叉熵损失函数，适用于多分类
optimizer = optim.Adam(model.parameters(), lr=learning_rate)  # Adam优化器
```

3. 模型训练与评估

(1) 训练过程执行

- 设置合适的训练轮数（epochs）和批量大小（batch size）。
- 将训练集数据输入网络，进行前向传播和反向传播。
- 根据损失函数的梯度更新网络的权重。
- 监控训练过程中的损失和准确率，以评估网络的训练效果。

```
for inputs, labels in train_loader:

    # 前向传播

    # 反向传播和优化
```

(2) 测试集评估模型

在测试集上评估模型性能，并保存相应的模型参数。

```
# 测试阶段
model.eval()
test_correct = 0
with torch.no_grad():

    test_acc = 100 * test_correct / len(test_loader.dataset) # 计算整个测试集的准确率
    test accuracies.append(test_acc)

# 每10个epoch打印一次训练和测试进度
if (epoch + 1) % 10 == 0:
    print(f'Epoch [{epoch + 1}/{num_epochs}], '
          f'Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%', ' # 显示训练损失和训练准确率
          f'Test Acc: {test_acc:.2f}%' ) # 显示测试准确率
```

(3) 结果分析与优化

- 尝试调整网络参数，如隐藏层数量、神经元数量、学习率等，以优化分类性能。
- 记录并比较不同参数设置下的实验结果
- 总结全连接神经网络在鸢尾花数据集上的分类性能。